

EcomAssistant — Backend

Error Handling Implementation Plan

July 2026

Project	EcomAssistant
Module	back/ (Express.js + TypeScript)
Current lines of code	~2,200 across 35+ files
Status	Analysis complete — ready to implement

Table of Contents

1 Executive Summary

2 Current Error Handling — Audit Results

2.1 What Works Well

2.2 Critical Gaps

2.3 Inconsistencies & Anti-Patterns

2.4 Priority Matrix

3 Implementation Phases

3.1 Phase 1 — Foundation (Day 1)

3.2 Phase 2 — Controller Refactor (Day 2)

3.3 Phase 3 — Service Layer Upgrade (Day 2-3)

3.4 Phase 4 — Polish & Observability (Day 3)

4 Code Specifications

4.1 AppError Class Hierarchy

4.2 Global Error Handler Middleware

4.3 asyncHandler Wrapper

4.4 Standardised Response Shape

4.5 Prisma Error Mapping

4.6 Logger Setup

4.7 File-by-File Changes

5 Risk & Mitigation

6 Acceptance Criteria

1 Executive Summary

This document outlines a structured plan to overhaul error handling in the EcomAssistant backend. The current implementation lacks a centralised error handler, relies on fragile string-matching for domain errors, has two controllers with no error handling at all, and returns inconsistent response shapes to clients.

The proposed changes introduce a typed error class hierarchy, a global Express error-handling middleware, an asyncHandler wrapper to eliminate boilerplate, structured logging, and centralised Prisma error mapping. The result will be resilient, observable, and maintainable error handling across the entire backend.

2 Current Error Handling — Audit Results

2.1 What Works Well

- Auth middleware:** Catches JWT errors (expired, invalid, revoked) with correct 401 responses and specific messages.
- Zod validation middleware:** Produces clean field-level error arrays with field and message keys.
- Rate limiting:** Basic express-rate-limit on signup, login, and OTP endpoints.
- CSRF protection:** Middleware validates double-submit cookie pattern for state-changing requests.
- Auth controller:** Maps known service errors to appropriate HTTP codes (409, 401, 429, 403, 404).
- BullMQ workers:** Log failures via failed event handlers, and re-throw so BullMQ can retry.

2.2 Critical Gaps

#	Issue	Severity	File(s)
1	No global Express error handler — every <code>next(err)</code> falls through to Express's default HTML handler, leaking stack traces.	Critical	Express's default HTML handler, leaking stack traces.
2	<code>listOrders</code> has no try/catch — a thrown error crashes the process or returns an unhandled rejection.	Critical	Order controller
3	<code>postFakeMessage</code> has no try/catch — same issue.	Critical	fakeMessages.controller.ts:5
4	Express 4 does not catch promise rejections in async route handlers automatically. Every async controller must wrap itself.	High	All controllers
5	Service errors are plain Error objects with magic strings. Controllers match on error.message + "!!". A typo silently breaks error handling.	High	Any service that breaks error handling.
6	Shopify controller never calls <code>next(err)</code> — it catches and returns <code>{ type: "Shopify", status: 500 }, json(...)</code> forever.	Critical	shopify.controller.ts
7	No Prisma error handling — unique constraint violations (P2002), connection drops, etc. are not caught or mapped.	High	Every prisma
8	Multiple error response shapes coexist across the codebase.	Medium	Various
9	Silent <code>catch {}</code> blocks in 6+ locations swallow errors with no logging.	Medium	whatsapp.controller.ts, shopify.controller.ts
10	No structured logging — everything is raw <code>console.log()</code> or <code>console.error()</code> .	Medium	Everywhere

2.3 Inconsistencies & Anti-Patterns

- **Error response shapes:** `{ message }` (auth), `{ error }` (shopify), `{ success: false, message, error }` (callback), `{ message, errors: [...] }` (validation).
- **Logging approach:** Some errors use `console.error + res.status(500)`, others `next(err)`, some do both.
- **Missing/null checks:** `req.merchant!` uses non-null assertion — if auth middleware were misconfigured or bypassed, this throws an unhelpful `TypeError`.
- **Swallowed catch blocks:** 6+ `catch {}` in `whatsapp.controller.ts` (QR polling, webhook cleanup, session cleanup, session status) and `shopify.controller.ts` (background syncs, webhook registration).

2.4 Priority Matrix

	High Impact	Low Impact
Easy	1. Global error handler 2. <code>asyncHandler</code> wrapper 3. Fix missing <code>try/catch</code> (2 files)	5. Standardise response shape 6. Logger setup
Hard	4. Custom error classes 5. Refactor service/controller error mapping	7. Review swallowed catch blocks 8. Remove non-null assertions

3 Implementation Phases

3.1 Phase 1 — Foundation (Day 1)

Goal: Establish the infrastructure that every other improvement depends on.

Deliverables

- **P1-A.** Create `src/lib/errors.ts` — `AppError` class hierarchy (see §4.1)
- **P1-B.** Create `src/middlewares/errorHandler.ts` — global Express error handler (see §4.2)
- **P1-C.** Create `src/middlewares/asyncHandler.ts` — async route wrapper (see §4.3)
- **P1-D.** Create `src/lib/logger.ts` — pino logger (see §4.6)
- **P1-E.** Wire global error handler + logger into `app.ts`

Files created: 4

Files modified: 1 (app.ts)

3.2 Phase 2 — Controller Refactor (Day 2)

Goal: Make every controller safe, consistent, and observable.

Deliverables

- **P2-A.** Wrap every async route handler with `asyncHandler`
- **P2-B.** Add missing `try/catch` to `listOrders` and `postFakeMessage`
- **P2-C.** Refactor Shopify controller to throw `AppError` subclasses and call `next(err)` instead of inline `res.status(500)`
- **P2-D.** Replace `console.error` in controllers with `logger.error(...)`
- **P2-E.** Standardise all error responses to `{ success: false, error: { code, message, details? } }`

Files modified:

- `auth.controller.ts`
- `orders.controller.ts`
- `whatsapp.controller.ts`
- `shopify.controller.ts`
- `fakeMessages.controller.ts`

3.3 Phase 3 — Service Layer Upgrade (Day 2–3)

Goal: Replace fragile magic-string errors with typed AppError subclasses in the service layer.

Deliverables

- **P3-A.** Replace `throw new Error("...")` in service files with typed errors:
 `throw new ConflictError("Email is already registered")`
 `throw new UnauthorizedError("Invalid Email")`
 `throw new NotFoundError("Merchant not found")`
 `throw new RateLimitError("Trop de tentatives...")`
- **P3-B.** Remove string-matching `if (error.message === ...) blocks` from controllers — let the global handler map by `error.constructor / error.code`
- **P3-C.** Add Prisma error mapping in the global handler (§4.5)

Files modified:

- `auth.service.ts` — ~18 throw statements
- `auth.controller.ts` — remove ~60 lines of string-matching
- `shopify.service.ts`
- `StoreConnectionFactory.ts`
- `ShopifyConnection.ts`
- `errorHandler.ts` — add Prisma mapping logic

3.4 Phase 4 — Polish & Observability (Day 3)

Goal: Eliminate noise, improve debuggability, and ensure long-term maintainability.

Deliverables

- **P4-A.** Audit all `catch {}` blocks — add `logger.warn(...)` or remove where appropriate
- **P4-B.** Replace `req.merchant!` non-null assertions with explicit null checks throwing `UnauthorizedError`
- **P4-C.** Add `NODE_ENV`-aware stack trace exposure in the error handler (show in development, hide in production)
- **P4-D.** Write integration tests for the global error handler
- **P4-E.** Run `tsc --noEmit` to verify type safety

Files modified:

- `whatsapp.controller.ts` — add logging to catch blocks
- `shopify.controller.ts` — add logging to catch blocks
- `errorHandler.ts` — stack trace control
- `auth middleware` — non-null assertion fix
- New: `errorHandler.test.ts`

4 Code Specifications

4.1 AppError Class Hierarchy

Create `src/lib/errors.ts`. All errors carry a `statusCode` (HTTP status), a `code` (machine-readable string for the client), and a human message. The global handler switches on `error.constructor` or `error.code`.

```
// src/lib/errors.ts
export class AppError extends Error {
  constructor(
    public statusCode: number,
    public code: string,
    message: string,
    public details?: unknown,
  ) {
    super(message);
    this.name = this.constructor.name;
  }
}
export class BadRequestError extends AppError {
  constructor(
    message = "Bad request",
    details?: unknown,
  ) {
    super(400, "BAD_REQUEST", message, details);
  }
}
export class UnauthorizedError extends AppError {
  constructor(
    message = "Unauthorized",
  ) {
    super(401, "UNAUTHORIZED", message);
  }
}
export class NotFoundError extends AppError {
  constructor(
    message = "Resource not found",
  ) {
    super(404, "NOT_FOUND", message);
  }
}
export class ConflictError extends AppError {
  constructor(
    message = "Resource already exists",
  ) {
    super(409, "CONFLICT", message);
  }
}
export class RateLimitError extends AppError {
  constructor(
    message = "Too many requests",
  ) {
    super(429, "RATE_LIMIT", message);
  }
}
export class InternalError extends AppError {
  constructor(
    message = "Internal server error",
  ) {
    super(500, "INTERNAL_ERROR", message);
  }
}
```

4.2 Global Error Handler Middleware

Create `src/middlewares/errorHandler.ts`. This 4-argument middleware is registered LAST in the Express middleware chain, after all routes. It catches every error that bubbles up via `next(err)`.

```
// src/middlewares/errorHandler.ts
import { Request, Response, NextFunction } from "express";
import { AppError } from "../lib/errors";
import { logger } from "../lib/logger";
import { Prisma } from "@prisma/client";
export function errorHandler(
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction,
): void {
  // 1. Log every error
  logger.error({ err, code: (err as any).code }, err.message);
  // 2. Known AppError subclasses
  if (err instanceof AppError) {
    res.status(err.statusCode).json({
      success: false,
      error: { code: err.code, message: err.message, ...(err.details ? { details: err.details } : {}),
        ...(process.env.NODE_ENV === "development" ? { stack: err.stack } : {}),
      },
    });
    return;
  }
  // 3. Prisma known errors
  if (err instanceof Prisma.PrismaClientKnownRequestError) {
    const prismaResponse = handlePrismaError(err);
    res.status(prismaResponse.statusCode).json({
      success: false,
      error: prismaResponse.error,
    });
    return;
  }
  // 4. Zod validation errors (defensive – should be caught earlier)
  if ((err as any).name === "ZodError") {
    res.status(400).json({
      success: false,
      error: {
        code: "VALIDATION_ERROR",
        message: "Validation failed",
        details: (err as any).errors?.map((e: any) => ({ field: e.path.join("."), message: e.message })),
      },
    });
    return;
  }
  // 5. Unknown / unexpected – return generic 500
  res.status(500).json({
    success: false,
    error: {
      code: "INTERNAL_ERROR",
      message: "An unexpected error occurred",
      ...(process.env.NODE_ENV === "development" ? { stack: err.stack } : {}),
    },
  });
}
```

4.3 asyncHandler Wrapper

Eliminates the try/catch boilerplate in every controller. Wraps an async Express handler so that any rejected promise automatically calls `next(err)`.

```
// src/middlewares/asyncHandler.ts import { Request, Response, NextFunction } from
"express"; type AsyncHandler = ( req: Request, res: Response, next: NextFunction, ) =>
Promise; export function asyncHandler(fn: AsyncHandler) { return (req: Request, res:
Response, next: NextFunction) => { Promise.resolve(fn(req, res, next)).catch(next);
}; }
```

Usage in routes: `router.get("/orders", asyncHandler(listOrders))`

4.4 Standardised Response Shape

Every API response will follow one of these two shapes:

```
// Success { "success": true, "data": { ... }, "pagination": { ... } // when
applicable } // Error { "success": false, "error": { "code": "NOT_FOUND", //
machine-readable "message": "Merchant not found", // human-readable "details": [ ...
] // optional, for validation errors } }
```

4.5 Prisma Error Mapping

Prisma throws `PrismaClientKnownRequestError` with a code property. Map the most common codes in the global handler:

```
const prismaErrorMap: Record = { P2000: { status: 400, code: "BAD_REQUEST" }, // value
too long P2002: { status: 409, code: "CONFLICT" }, // unique constraint P2025: {
status: 404, code: "NOT_FOUND" }, // record not found P2003: { status: 400, code:
"FOREIGN_KEY_VIOLATION" }, // FK violation }; function handlePrismaError(err:
PrismaClientKnownRequestError) { const mapping = prismaErrorMap[err.code]; if
(mapping) { return { statusCode: mapping.status, error: { code: mapping.code,
message: err.message.replace(/\n/g, " "), ...(err.meta ? { details: err.meta } : {}),
}, }; } // Fallback for unhandled Prisma codes return { statusCode: 500, error: {
code: "DATABASE_ERROR", message: "Database operation failed" }, }; }
```

4.6 Logger Setup

Replace all `console.log/console.error` with `pino`, a low-overhead structured logger. This gives us JSON output in production, pretty-print in development, and automatic inclusion of request IDs, timestamps, etc.

```
// src/lib/logger.ts import pino from "pino"; const isDev = process.env.NODE_ENV !==
"production"; export const logger = pino({ level: process.env.LOG_LEVEL ?? (isDev ?
"debug" : "info"), ...(isDev ? { transport: { target: "pino-pretty", options: {
colorize: true, translateTime: "HH:MM:ss" }, }, } : {}), });
```

Install: `pnpm add pino && pnpm add -D pino-pretty`

4.7 File-by-File Changes

Detailed breakdown of every file that will be created or modified:

File	Action	Description
src/lib/errors.ts	CREATE	AppError class hierarchy (7 classes)
src/lib/logger.ts	CREATE	Pino logger instance
src/middlewares/errorHandler.ts	CREATE	Global Express error handler with Prisma mapping
src/middlewares/asyncHandler.ts	CREATE	Async route wrapper
src/app.ts	MODIFY	Import and register error handler last, import logger
src/modules/auth/auth.controller.ts	MODIFY	Remove string-matching if-blocks; wrap handlers with asyncHandler; replace console.error → logger.error
src/modules/auth/auth.service.ts	MODIFY	Replace throw Error → throw typed AppError subclasses
src/modules/orders/orders.controller.ts	MODIFY	Add asyncHandler wrapper; add try/catch
src/modules/whatsapp/whatsapp.controller.ts	MODIFY	Wrap with asyncHandler; add logging to catch {} blocks; replace console.error → logger.error
src/modules/whatsapp/whatsapp.service.ts	MODIFY	Replace OpenwaError extends Error → OpenwaError extends AppError (optional)
src/modules/storeConnections/shopify/shopify.controller.ts	MODIFY	Replace inline res.status(500) → next(err); wrap with asyncHandler; replace console.error → logger.error
src/modules/storeConnections/shopify/shopify.service.ts	MODIFY	Replace throw Error → throw typed errors
src/modules/fakeMessages/fakeMessages.controller.ts	MODIFY	Add asyncHandler wrapper; add try/catch
src/connections/StoreConnectionFactory.ts	MODIFY	Replace throw Error → throw typed errors
src/connections/ShopifyConnection.ts	MODIFY	Replace throw Error → throw typed errors
src/middlewares/auth.middleware.ts	MODIFY	Replace req.merchant! with explicit null check; replace console.error → logger.error

5 Risk & Mitigation

Risk	Likelihood	Impact	Mitigation
Refactoring service throws changes in controller behaviour	Medium	High	Write/run integration tests before and after each phase. Keep string-matching fallback in place.
pino adds noticeable latency in hot paths	Low	Medium	pino is benchmarked as one of the fastest Node.js loggers (~2µs per call). Use 'info' level for most logging.
Merging conflicts with in-flight feature branches	Medium	Medium	Phase 1 (new files) has zero conflict risk. Phase 2-3 touch shared files — coordinate with team.
Global handler accidentally swallows expected errors	Low	High	Add a unit test suite for errorHandler that verifies mapping for every AppError subclass.
Missing Prisma error code in mapping	Medium	Low	Unmapped codes fall through to a 500 generic response — safe by default. Add monitoring for 500s.

6 Acceptance Criteria

- AC-1: Every async route handler is wrapped with `asyncHandler` or has an explicit `try/catch` calling `next(err)`.
- AC-2: The global error handler is registered last in the Express middleware chain and returns JSON for every error.
- AC-3: Every service file throws typed `AppError` subclasses instead of `Error`.
- AC-4: No controller file matches on `error.message` strings.
- AC-5: All API error responses follow the shape `{ success: false, error: { code, message } }`.
- AC-6: Prisma P2002 returns 409, P2025 returns 404, P2000 returns 400.
- AC-7: `listOrders` and `postFakeMessage` have error handling.
- AC-8: No `console.log/console.error` remains — all logging uses the pino logger.
- AC-9: Stack traces are exposed in development but hidden in production.
- AC-10: All silent `catch {}` blocks either log a warning or are justified with a comment.
- AC-11: `tsc --noEmit` passes without errors.
- AC-12: The global error handler has unit tests covering all `AppError` subclasses and all mapped Prisma error codes.

— End of Document —